7. Define a variable. Explain declaration and initialization of a variable.

Ans :

a) Definition:

A **variable** is a storage location identified by a name, and **initialization** assigns a value to it.

b) Types of Variable Declaration:

Explicit Initialization – Assigns a value at declaration.

Syntax: int x = 10;

Example:

```
int score = 95;
System.out.println("Score: " + score);
```

Dynamic Initialization – Uses an expression for initialization.

Syntax: dataType var = expression;

Example:

```
int a = 5, b = 10;
int sum = a + b;
System.out.println("Sum: " + sum);
```

Multiple Variable Declaration – Declares multiple variables of the same type.

Syntax: int a, b, c;

Example:

```
int x = 1, y = 2, z = 3;
System.out.println("Values: " + x + ", " + y + ", " + z);
```

8. Explain Scope and lifetime of variables with an example.

Ans :

a) Definition:

Scope defines **where a variable is accessible**, and lifetime determines **how long it exists**.

b) Types of Scope:

Local Scope – Variable exists only inside a method or block.

Syntax:

```
void myMethod() {
    int localVar = 10; // Only accessible inside this method
}
```

Example:

```
public class Example {
    public static void main(String[] args) {
        if (true) {
            int x = 100; // x is local to this block
            System.out.println("Value: " + x);
        }
        // x is not accessible here
    }
}
```

Instance Scope – Variables belong to an object, accessible using this.

Syntax:

```
class MyClass { int instanceVar; }
```

Example:

```
class Car {
    int speed; // Instance variable
}
```

Static Scope – Shared among all instances of a class.

Syntax:

```
static int count = 0;
```

Example:

```
class Counter {
    static int count = 0; // Shared by all instances
}
```

The Bitwise AND

The AND operator & produces a 1 bit if both operands are also 1.
A zero is produced in all other cases.

Example :

	1 Byte								1 Byte								1 Byte								1 Byte							
8	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
15	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1
8&15 = 8	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0

The Bitwise OR

The OR operator | combines bits such that if either of the bits in the operands is a 1, then the resultant bit is a 1.

Example :

	1 Byte								1 Byte								1 Byte								1 Byte							
8	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
15	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1
8^15 =7	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1

The Bitwise XOR

The XOR operator ^ combines bits such that if exactly one operand is 1, then the result is 1.Otherwise, the result is zero. The following example shows the effect of the ^. This example also... a useful attribute of the XOR operation. Notice how the bit pattern of 42is inver... second operand has a 1 bit. Wherever the second operand has a 0 bit, the first oper...

21 / 60

	1 Byte								1 Byte								1 Byte								1 Byte							
8	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
15	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1
8^15 =7	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1

Example:

```
int a=8,b=15, res1, res2, res3, res4;

res1=~a;
res2=a & b;
res3=a | b;
res4 = a^b;

System.out.println("~a= "+res1);
System.out.println("a & b= "+res2);
System.out.println("a | b= "+res3);
System.out.println("a ^ b= "+res4);
```

Output:
~a= -9
a & b= 8
a | b= 15
a ^ b= 7

The Left Shift

The left shift operator << shifts all of the bits in a value to the left a specified number of times. It has this general form: *value* <<*num*

Example :

5 << 1 means 5 is shifted towards left by 1 bit position = 10

5	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1
5<<1	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0

7<<2 means 7 is shifted towards left by 2 bit positions = 28

7	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1
7<<2	0	0	0	0	0	0	0	0	0	0	0	1	1	1	0	0

```
byte a = 5, b;
b = (byte) (a << 1);
```

The Bitwise AND

The AND operator & produces a 1 bit if both operands are also 1.
A zero is produced in all other cases.

Example :

	1 Byte								1 Byte								1 Byte								1 Byte							
8	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
15	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1
8&15 =8	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0

The Bitwise OR

The OR operator | combines bits such that if either of the bits in the operands is a 1, then the resultant bit is a 1.

Example :

	1 Byte								1 Byte								1 Byte								1 Byte							
8	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
15	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1
8^15 =7	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1

The Bitwise XOR

The XOR operator ^ combines bits such that if exactly one operand is 1, then the result is 1.Otherwise, the result is zero. The following example shows the effect of the ^. This example also demonstrates a useful attribute of the XOR operation. Notice how the bit pattern of 42is inverted wherever the second operand has a 1 bit. Wherever the second operand has a 0 bit, the first operand is unchanged.

	1 Byte								1 Byte								1 Byte								1 Byte							
8	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
15	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1
8^15 =7	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1

Example:

```
int a=8,b=15, res1, res2, res3, res4;

res1=~a;
res2=a & b;
res3=a | b;
res4 = a^b;

System.out.println("~a= "+res1);
System.out.println("a & b= "+res2);
System.out.println("a | b= "+res3);
System.out.println("a ^ b= "+res4);
```

Output:
```
~a= -9
a & b= 8
a | b= 15
a ^ b= 7
```


The Left Shift

The left shift operator << shifts all of the bits in a value to the left a specified number of times. It has this general form: *value* <<*num*

Example :

5 << 1 means 5 is shifted towards left by 1 bit position = 10

5	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1
5<<1	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0

7<<2 means 7 is shifted towards left by 2 bit positions = 28

7	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1
7<<2	0	0	0	0	0	0	0	0	0	0	0	1	1	1	0	0

```
byte a = 5, b;
b = (byte) (a << 1);
```

```
System.out.println( Left Shift by 1 bit );
System.out.println("a= "+ a + " b=" + b);
a = 7;
b = (byte) (a << 2);
System.out.println("Left Shift by 2 bits");
System.out.println("a= "+ a + " b=" + b);
```

The Signed Right Shift

The right shift operator, >>, shifts all of the bits in a value to the right a specified number of times. It has the general form : value >>num

Example :

5 >> 1 means 5 is shifted towards right by 1 bit position = 2

5	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1
5>>1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0

12>>2 means 12 is shifted towards left by 2 bit positions = 3

12	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0
12>>2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1

Example :

```
byte a = 5, b;
b = (byte) (a >> 1);
System.out.println("Right Shift by 1 bit");
System.out.println("a= "+ a + " b=" + b);
a = 12;
b = (byte) (a >> 2);
```

The if statement is a **control flow statement** used to execute a block of code **only if a specified condition is true**.

b) Types of if Statements:

1. **Simple if Statement** – Executes a block of code **if the condition is true**.
   - **Syntax:**

```
if (condition) {
  // Code executes if condition is true
}
```

Example:

```
int x = 10;
if (x > 5) {
  System.out.println("x is greater than 5");
}
```

2. **if-else Statement** – Executes **one block if true**, another if false.

Syntax:

```
if (condition) {
  // Executes if true
} else {
  // Executes if false
}
```

Example:

```
int num = 10;
if (num % 2 == 0) {
  System.out.println("Even Number");
} else {
  System.out.println("Odd Number");
}
```

3. **Nested if Statement** – An if statement **inside another if statement**.

Syntax:

```
if (condition1) {
  if (condition2) {
    // Executes if both conditions are true
  }
}
```

Example:

```
int age = 20;
if (age > 18) {
  if (age < 60) {
    System.out.println("Eligible for work");
  }
```

BPLCK205C

USN | | | | | | | | | | |

ADITYA COLLEGE OF ENGINEERING & TECHNOLOGY

Affiliated to VTU, Belagavi, Approved by AICTE, New Delhi

II SEMESTER INTERNAL ASSESSMENT TEST- I

Course Title	Basics of Java Programming	**Maximum Marks**	25
Date and Time	26-04-2025, 10.00 AM- 11.00 AM	**No. of Hours**	1
Department	CSE and allied Branches		

Instructions to Students:

1. Answer **TWO QUESTIONS** full selecting one question from each part.
2. Possession of any kind of written material, mobile/electronic gadgets & scribbling on QP, amounts to malpractice.
3. Any missing data may assume suitably.

Q. No.		Questions	Max Marks	CO	RBTL
		PART A (Module - 1, 10 Marks)			
1	a	Explain Three OOP Principles	**5**	1	L2
	b	Explain Primitive data Types	5	1	L2
		OR			
2	a	Explain the Scope and Lifetime of Variables with a program for each .	5	1	L2
	b	Explain the following operators with example And(&) Or(|) Right shift(>>) Left Shift(<<)	5	2	L3
		PART B (Module - 2, 15 Marks)			
3	a	Explain 4 different types of if statement in java	7	2	L2
	b	Explain while and do while loop with an example	8	2	L3
		OR			
4	a	Explain the switch statement with the program	7	2	L2
	b	Explain jump statement with an example	8	3	L3

Faculty Signature	HOD Signature